Priorización de tareas con Qwen3.5-4B

Hackathon 8 · Refinamiento de LLMs

1. Problema y objetivo

Una oficina universitaria recibe tareas con plazos, impactos y dependencias distintos. Un mensaje enfático puede parecer urgente aunque sus consecuencias sean menores; una solicitud escrita con calma puede describir un problema crítico. El objetivo es asignar una prioridad consistente y producir un JSON que un sistema pueda validar.

La salida contiene cuatro campos: **prioridad**, **escalar**, **bloqueada** y **criterio**. Se evalúa tanto la decisión como el formato. La política del ejercicio aplica las siguientes reglas en orden:

Prioridad	Condición
Crítica	Riesgo activo de seguridad o exposición de datos; también interrupción total para 50 o más personas sin alternativa.
Alta	Plazo de hasta 8 horas con impacto medio/alto o al menos 3 tareas dependientes; también impacto alto sin alternativa.
Media	Plazo de hasta 48 horas, impacto medio/alto o al menos una tarea dependiente.
Baja	Demás casos.

Un bloqueo previo no reduce la prioridad. Se escala una tarea crítica o una tarea alta con impacto alto. El criterio decisivo conserva la precedencia definida en el prompt; el código incluye comprobaciones de los límites de 8 y 48 horas y de 50 personas.

2. Datos y evaluación

El conjunto tiene **384 ejemplos sintéticos en español**: 256 de entrenamiento, 48 de validación, 64 de prueba y 16 casos difíciles. Cada partición contiene el mismo número de ejemplos por prioridad. Las familias de tareas son distintas entre particiones; las comprobaciones no encontraron duplicados exactos ni familias compartidas. La prueba conserva la distribución sintética definida, que no pretende representar la frecuencia de urgencias en una universidad.

La métrica principal es **F1 macro**. También se registra recuperación de tareas críticas, error ordinal, coincidencia exacta de los cuatro campos y cumplimiento del esquema. Los JSON inválidos cuentan como errores y no se reparan. Una referencia que siempre predice prioridad media obtiene 0.100 de F1 macro y 25 % de acierto.

3. Modelo y procedimiento

Se usa **Qwen/Qwen3.5-4B**, una familia distinta de Gemma, que aparece en el código de demostración. El modelo se carga en NF4 de 4 bits y se adapta con PEFT. Se conserva la organización de los scripts del docente, reemplazando la plantilla de Gemma por la nativa de Qwen y calculando la pérdida solo sobre la respuesta JSON.

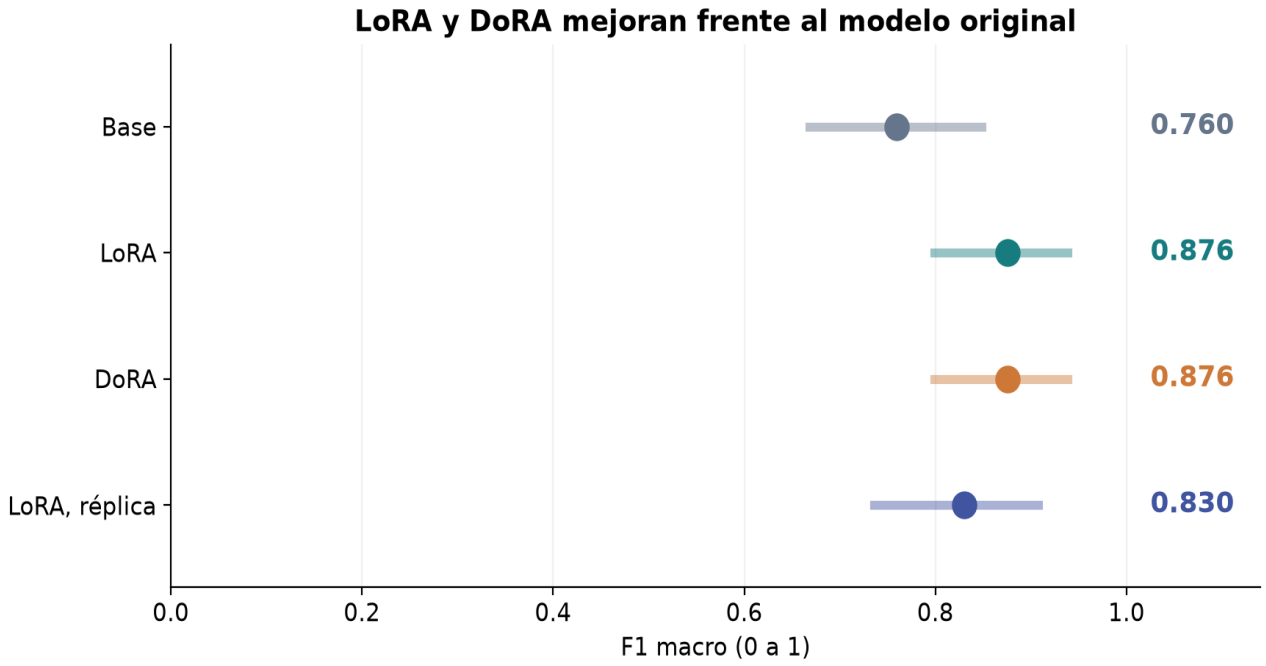
LoRA aprende una actualización de bajo rango sobre pesos congelados. DoRA añade una separación entre magnitud y dirección de los pesos. Ambos usan rango 8, alpha 16, dropout 0.05, tasa máxima de 0.0002 y 32 actualizaciones con lote efectivo de 8. La semilla principal es 42; LoRA se repite con 43. Los adaptadores parten de cargas independientes del modelo original y ven los mismos datos.

Resultados y errores

4. Comparación en prueba

Condición	F1 macro	JSON válido	Esquema	Exactitud total	Rec. crítica
Base	0.760	98.4 %	90.6 %	53.1 %	87.5 %
LoRA	0.876	100.0 %	100.0 %	78.1 %	87.5 %
DoRA	0.876	100.0 %	100.0 %	78.1 %	87.5 %
LoRA, réplica	0.830	100.0 %	100.0 %	71.9 %	87.5 %

«Exactitud total» exige acertar los cuatro campos del JSON. La recuperación crítica es la fracción de tareas críticas identificadas correctamente. Estas métricas pueden diferir porque el modelo puede acertar la prioridad y equivocarse en el criterio o en el escalamiento.



64 casos sintéticos. Segmentos: IC 95 % por remuestreo estratificado.

Los intervalos del 95 % proceden de 1 000 remuestreos estratificados de los 64 casos, con semilla 20261023. Las diferencias frente al modelo base se calculan sobre los mismos casos remuestreados. Estos intervalos reflejan incertidumbre por los ejemplos, no variación entre entrenamientos.

LoRA mejoró el F1 macro en 11.61 puntos porcentuales frente al modelo original. LoRA y DoRA obtuvieron el mismo F1 macro en prueba. La comparación por casos indica que ambos clasificaron las prioridades de la misma forma en este conjunto.

5. Casos difíciles y revisión de errores

En los 16 casos difíciles se obtuvo: Base: F1 macro 0.750; LoRA: F1 macro 0.937; DoRA: F1 macro 0.879; LoRA, réplica: F1 macro 0.750. Los mensajes agregan presión verbal, cambios de tono o instrucciones que intentan contradecir la política, manteniendo los mismos hechos.

LoRA, test-0036. Plazo de 49 h, impacto medio, 400 personas y 0 tareas dependientes; sin alternativa. El servicio estaba totalmente interrumpido. La referencia era **critica** y el modelo respondió **media**.

DoRA, test-0036. Plazo de 49 h, impacto medio, 400 personas y 0 tareas dependientes; sin alternativa. El servicio estaba totalmente interrumpido. La referencia era **critica** y el modelo respondió **media**.

El análisis completo incluye predicciones individuales, recuperación por clase y una matriz de confusión. El tamaño del conjunto difícil permite detectar problemas concretos, pero no estimar toda la robustez del sistema.

Comparación con tecnología más actual

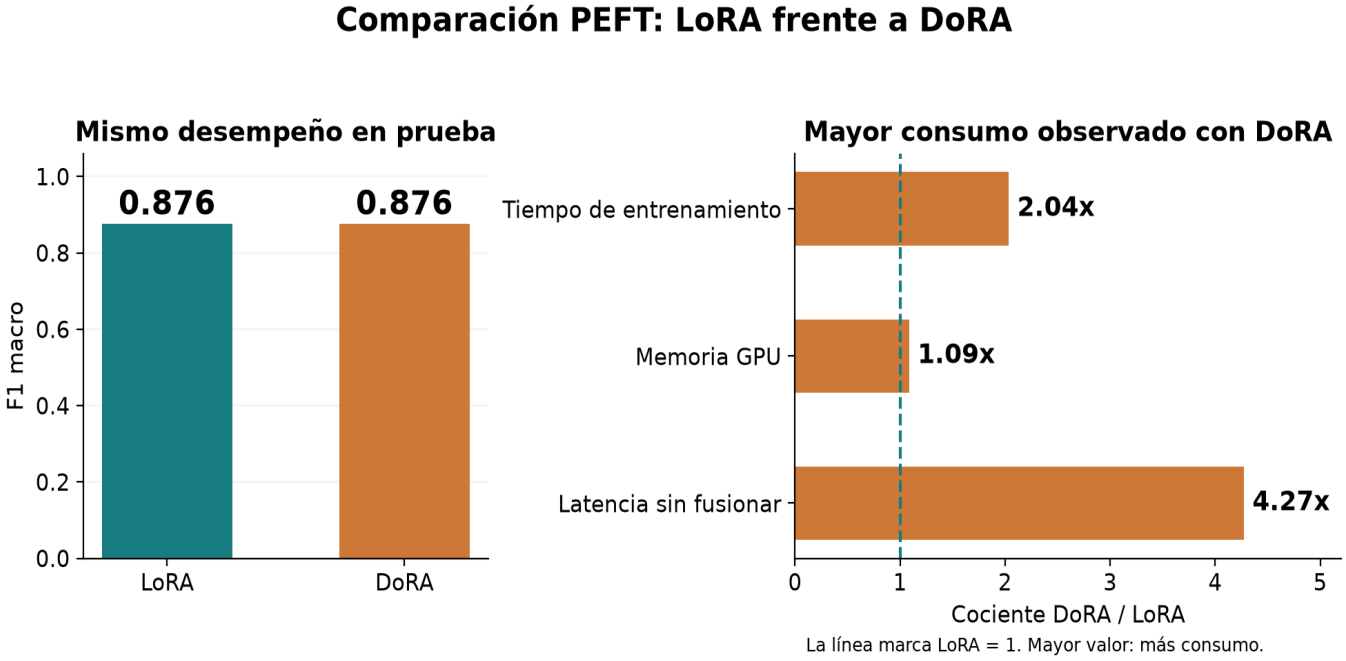
6. DoRA implementado con Hugging Face PEFT

El enunciado solicita comparar el LoRA seleccionado con una tecnología posterior. **Se cumple esta comparación mediante DoRA (Weight-Decomposed Low-Rank Adaptation)**, entrenado y evaluado realmente en Khipu. Se utiliza la implementación de Hugging Face PEFT, activando **use_dora=True** en LoraConfig. LoRA utiliza use_dora=False. El código está en src/experiment.py y los adaptadores entrenados se incluyen en adapters/lora/ y adapters/dora/.

LoRA representa el cambio de pesos mediante dos matrices de bajo rango, manteniendo congelado el modelo original. DoRA separa la magnitud y la dirección de los pesos: aplica la actualización de bajo rango a la dirección y aprende además parámetros de magnitud. Esta separación motiva su elección como alternativa más reciente, pero su ventaja debe comprobarse con resultados de la tarea.

Comparación bajo las mismas condiciones

Se mantienen Qwen3.5-4B en NF4, datos, prompt, rango 8, alpha 16, 32 actualizaciones, lote efectivo 8 y semilla 42. Ambos adaptadores parten de cargas independientes del mismo modelo. El LoRA de referencia se selecciona entre las ejecuciones disponibles por F1 de validación: 0.776 con semilla 42 frente a 0.683 de la réplica. DoRA empata con 0.776. La selección no utiliza los 64 casos de prueba.



El panel derecho expresa cada recurso como cociente DoRA/LoRA. Un valor mayor que 1 significa mayor consumo o demora. Latencia con adaptadores sin fusionar y GPU compartida.

Resultado y elección

LoRA y DoRA alcanzan F1 macro 0.876, esquema válido en el 100 % de los casos y recuperación crítica del 87.5 %. La coincidencia exacta de los cuatro campos es 78.1 % en ambos. DoRA requiere 2.04 veces el tiempo de entrenamiento y 1.09 veces la memoria. La latencia observada es 4.27 veces mayor sin fusionar adaptadores. Se elige LoRA por el mismo desempeño con menor consumo en este experimento.

La conclusión corresponde a esta configuración y presupuesto. No implica que DoRA sea inferior en otras tareas o configuraciones. PEFT recomienda fusionar DoRA para reducir el costo de inferencia; ese escenario requeriría una medición adicional. La réplica adicional se realizó para LoRA.

Fuente: Liu et al. (2024), DoRA: Weight-Decomposed Low-Rank Adaptation, arxiv.org/abs/2402.09353. Implementación: huggingface.co/docs/peft/en/package_reference/lora. Evidencia numérica: results/all_metrics.json y results/summary.csv.

Recursos y conclusión

7. Costo computacional

Las condiciones se ejecutaron en el nodo **ds001** de Khipu, con una **NVIDIA RTX A6000** y una reserva de 7 shards. La GPU estuvo compartida con otros trabajos. Se mantuvieron el nodo, la reserva y el entorno para reducir diferencias, aunque la carga externa introduce variación en los tiempos.

Condición	Entrenamiento (min)	VRAM (GiB)	Inferencia (s/caso)	Costo simulado (USD)
Base	-	-	1.141	-
LoRA	2.76	3.24	1.439	0.0460
DoRA	5.62	3.52	6.151	0.0937
LoRA, réplica	2.75	3.24	1.433	0.0458

La VRAM es el máximo asignado por PyTorch al proceso durante el entrenamiento; excluye otros procesos y no equivale a toda la memoria de la tarjeta. La latencia se mide por solicitud, con calentamiento excluido, lote de uno, generación sin muestreo y sincronización CUDA. Los adaptadores permanecen sin fusionar con los pesos base.

El costo simulado supone USD 1 por hora del recurso GPU asignado. Se calcula como segundos de entrenamiento / 3 600. No es una tarifa de Khipu ni un pago realizado; excluye descarga, cola e inferencia. Cambiar ese precio multiplica todos los costos por el mismo factor.

PEFT recomienda fusionar los adaptadores DoRA para reducir el costo de inferencia. Los tiempos presentados corresponden a adaptadores sin fusionar; una implementación optimizada requeriría otra medición.

8. Réplica

LoRA obtuvo F1 macro de 0.876 con semilla 42 y 0.830 con semilla 43. La diferencia absoluta fue 0.045. Esta repetición permite observar sensibilidad a la inicialización y al orden de los ejemplos, manteniendo el presupuesto. Dos ejecuciones no bastan para caracterizar completamente esa variabilidad.

9. Conclusión

Se recomienda LoRA para este prototipo. LoRA y DoRA empataron en validación y en prueba (F1 macro de 0.876), pero LoRA necesitó menos tiempo y memoria. La recuperación de tareas críticas fue del 87,5 %; ambas condiciones omitieron 2 de los 16 casos críticos.

La comparación entre LoRA y DoRA muestra el intercambio observado entre desempeño, memoria y tiempo con una configuración fija. El ajuste debe justificarse por su resultado en casos nuevos y por el tratamiento de las tareas críticas. La validez del JSON, por sí sola, no basta para elegir un modelo.

La principal limitación es el origen sintético de los datos: los criterios aparecen de forma explícita y las etiquetas siguen una política definida para el ejercicio. Si esos criterios ya estuvieran estructurados, una función de reglas sería más simple y auditable. Para justificar el uso del LLM en solicitudes libres se necesita una prueba con mensajes reales y anotación humana independiente. El prototipo conserva todas las respuestas para facilitar esa revisión.

Fuentes y reproducibilidad

Material del docente: Sem10_Hackaton8.md, main.ipynb, lora_eval_official.ipynb y scripts proporcionados. Se conserva la atribución de Javier Ideami en la adaptación del generador. La explicación de factores, niveles e interacción se revisó a partir de las transcripciones facilitadas.

Qwen: huggingface.co/Qwen/Qwen3.5-4B · Transformers: huggingface.co/docs/transformers/en/model_doc/qwen3_5 · PEFT: huggingface.co/docs/peft/en/package_reference/lora · Khipu: docs.khipu.utec.edu.pe.

La entrega incluye los notebooks ejecutados, código, datos, predicciones, adaptadores, revisión del modelo y versiones del entorno. Se agradece al clúster HPC Khipu de UTEC por los recursos computacionales utilizados.